

Red Spotify

SOFTWARE DESIGN DOCUMENT (SDD)

CS 4308 Senior Project

Spring Semester 2026

Professor Perry

02/08/2026

Table of Contents

1.	INTRODUCTION AND OVERVIEW.....	3
2.	DESIGN CONSIDERATIONS.....	3
2.1.	ASSUMPTIONS AND DEPENDENCIES.....	3
2.2.	GENERAL CONSTRAINTS	3
2.3.	DEVELOPMENT METHODS.....	3
3.	ARCHITECTURAL STRATEGIES.....	4
4.	SYSTEM ARCHITECTURE.....	4
4.1.	FLOW DIAGRAM	5
5.	DETAILED SYSTEM DESIGN	6
5.1.	CLASSIFICATION.....	6
5.2.	DEFINITION.....	6
5.3.	CONSTRAINTS.....	6
5.4.	RESOURCES	6
5.5.	INTERFACE/EXPORTS.....	7
6.	GLOSSARY	7
7.	BIBLIOGRAPHY	7
8.	TUTORIAL SCREENSHOTS.....	8

1. Introduction and Overview

Red Spotify is a mobile software application designed to improve user personalization, music discovery, and user music data insight by using Spotify's existing Web API. Spotify accomplishes a great extensive music library available to be streamed at any time, however it offers little customization of the user's music data analytics and the social visibility of user's listening habits. Red Spotify will address this gap by giving users curated music discovery software features, listening habit analytics, and a modern UI design.

Red Spotify integrates the Spotify Web API to gain access to user listening data, playlist, playback features, and metadata. It is developed using React Native to support both Android and iOS use. This document was made to describe the architectural design and strategies that are used in the implementation of Red Spotify.

2. Design Considerations

2.1. *Assumptions and Dependencies*

The design of Red Spotify is based on the following assumptions and dependencies:

- Spotify OAuth is required for user authentication
- Users must have a Spotify account
- Internet connectivity is required
- Modern mobile Android or iOS devices
- Spotify Web API is available

2.2. *General Constraints*

The design of Red Spotify is based on these global constraints:

- **Authentication Constraints:** User authentication must follow Spotify OAuth terms and policies
- **Platform Constraints:** App must run on both iOS and Android using the shared React Native codebase
- **API Constraints:** Spotify Web API rate and data access limits
- **Testing Constraints:** Playback and data come from Spotify Web API, so testing is only as valid as Spotify accounts and API availability.
- **Performance Constraints:** Responsiveness of UI has to remain despite large music libraries and data
- **Security Constraints:** Communication will occur using HTTPS

2.3. *Development Methods*

This project follows the Agile methodologies. The project is divided smaller phases including research, UI prototypes, API integration, feature implementation, and lastly testing. This approach allows for completion of tasks and also allows for changes of features scope later on in the timeline.

3. Architectural Strategies

React Native and Expo

We will be using React Native to build the app since it offers a lot of flexibility and options in app design. Since we want the app for IOS and Android, we will have to use a framework so that everyone can test for different devices on their own, and since it is recommended for React projects anyway. We will likely use Expo since it's the recommended framework by React Native.

Error Detection

In the case where information is not available to the API, the app will catch the exception and inform the user that the information is not available. If the information is required for an element on the dashboard, the app will either remove the element (if all info is missing) or replace the specific text with "unavailable."

Memory and Data Storage

All data will be stored locally on the user's device. There was some consideration for allowing users to interact with other profiles, but the scope for the project would likely be too large for our time frame if we chose to implement that along with everything else.

Spotify Synchronization

The app will require internet access for use, as the Spotify API will not work without it. While Spotify allows listening to downloaded content offline, there would not be any features that are not already available on Spotify. For this reason, there is no point creating a version of the app that works without internet connection.

Song Search

Since both the user and Spotify libraries will have search functions, there will be a specific search method that is developed and then implemented in both; the only difference will be the pool of songs to search from. This will allow us to apply the same function to every search action; the alternative would be to code an individual search function for every feature that uses it.

4. System Architecture

4.1. Components

The components will be split based on functionality: the main components will include a Current Track element, Search element, Library viewing and editing element, and a Personal Dashboard that contains user info. These components will be split into pages that the user can navigate to using the bottom of the app page.

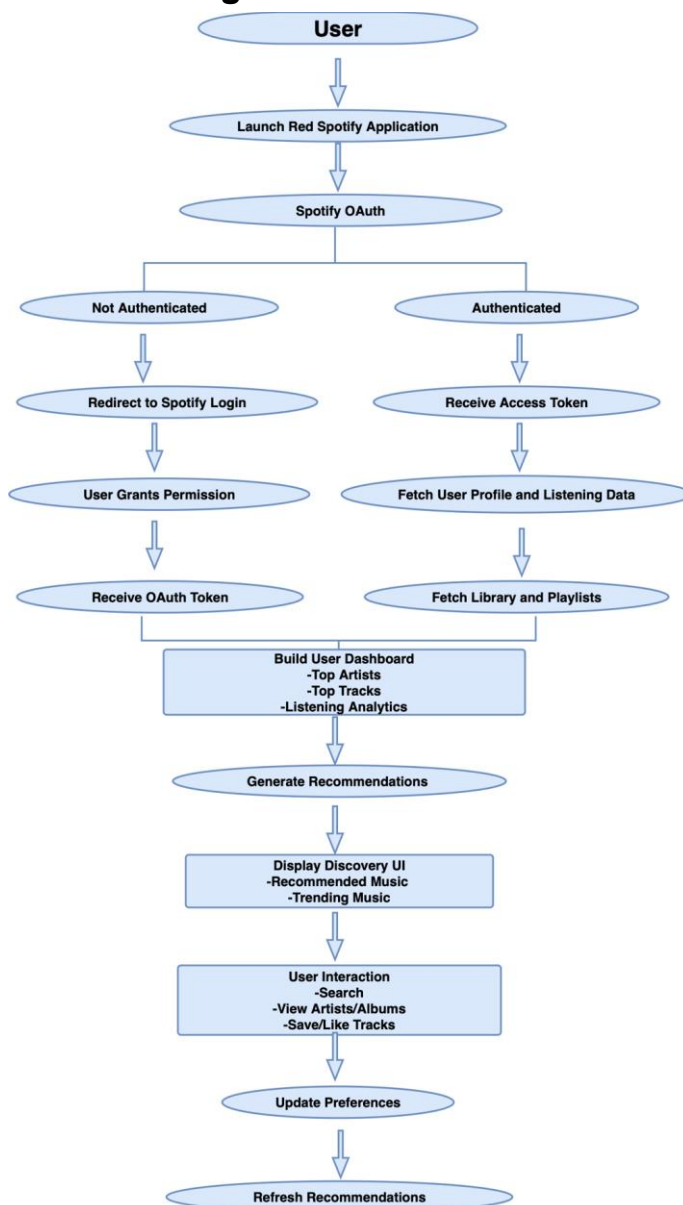
Personal Dashboard

This will contain all the listening info for the user and organize it on a dashboard.

Active Track

Displays the track the user is currently listening to; for everything related to active track.

4.2. Flow Diagram



5. Detailed System Design

5.1. *Personal Dashboard*

5.1.1. Classification

This component will be written as a class.

5.1.2. Definition

The dashboard will provide the user with their Spotify listening information. It will take the form of multiple, reorganizable boxes containing different elements. It will provide information like top artists, top songs, recently played, minutes listened, and shows near the player's city.

5.1.3. Constraints

Certain information may not always be available depending on the user's activity and Spotify API's anti AI constraints. The dashboard will need to compensate in some way when information is not available to view.

5.1.4. Resources

This entity will need to access certain features of Spotify API: Albums (saved albums, new releases), Audiobooks (saved books), Episodes (saved episodes), Shows, Users

5.1.5. Interface/Exports

The component will separate each element into their own subclasses with individual info-retrieval methods. There will be a single element class that controls features that every element has (size, font, etc.), while the subclasses will contain the methods to get their specific information and format it.

The dashboard itself will be organized by building the element blocks on a grid and fitting them together. This will require a grid type where each element is a unit inside of the grid.

5.2. *Active Track*

5.2.1. Classification

The component will be a class.

5.2.2. Definition

The component will control all features related to the active track for the user. It will provide info related to the track (artist, album, length, bpm, key) and allow the user to access playback (play, pause, skip, queue, etc.).

5.2.3. Constraints

Since Spotify API has recently restricted the ability to access certain info about the track, we will use another third-party API for info related to the songs' BPM, key, or other features. We will likely use GetSongKey API for this. The API is free to use, but does require a backlink to work, so that will be included.

5.2.4. Resources

This entity will need to access certain features of Spotify API: Track, Player, Artists (get artist), Albums (get Album), Chapters, Episodes (get episode), Users (check if user follows artist, follow/unfollow artist), Playlists (add/remove items)

5.2.5. Interface/Exports

Provides users with the ability to play, pause, skip, rewind, set mode, set volume, seek position, view queue, edit queue, and view currently playing track. Also shows user the artist and album and allows them to visit either.

5.3. Library

5.3.1. Classification

This component will be a subsystem of elements. Each playlist, artist, and album will function as an element in the standard library. The library as an entity will be a class comprised of every track saved to the user's account. The component will include a search method and a sort method. There will be an overarching search method to search for albums and artists.

Definition

The component will represent the page that shows all of the user's library information. The library page will look similar to the Spotify library page, but there will also be a library playlist. The playlist will be a list of every track in the user's library, where they will be able to sort and search (local files included). This component will also include methods for editing and sorting regular playlists.

Constraints

We will be operating off of the limitations of the Spotify API. Since Spotify no longer allows viewing of audio features, we can't sort based off of anything contained in the audio of the track.

Resources

This entity will need to access certain features of Spotify API: Albums (get album), Artists (get artist), Playlists, Search, Tracks (get users saved tracks, save tracks, remove saved tracks)

Interface/Exports

This component will provide the ability to edit and sort playlists, edit and sort your full library, search your library, and remove items from your library. Each of these features will be presented as a different method that can be called by user action.

Search

Classification

This component will technically be a class (to represent the page), but all of the substance of this component will essentially just be a collection of methods.

Definition

This component will mimic the regular Spotify search function, but it will allow the user to add specific filters to the search.

5.3.2. Constraints

We will be using the Spotify API search function, so everything will have to be sorted using the provided output of that function.

5.3.3. Resources

This component will need to use the Spotify API search. It will also need access to artist, album, and track information, as well as user saves to show what is already in the user's library.

5.3.4. Interface/Exports

This component will provide the user with the ability to search and filter artists, tracks, and albums on Spotify. Along with functioning like the Spotify Search, it will also filter the searched content based on user filter selections. When the user performs a filtered search, the program will take the results and remove them as they load if they do not match the criteria.

6. Glossary

1. API (Application Programming Interface)
 - a. A set of rules that allows one software application to interact with another
2. Caching
 - a. Temporary storage of frequently accessed data
3. HTTPS (Hypertext Transfer Protocol Secure)
 - a. A secure communication protocol used to encrypt data transmitted between the application and external services
4. OAuth (Open Authorization)
 - a. A protocol for secure authorization
5. UI (User Interface)
 - a. The visual and interactive elements of an application
6. UX (User Experience)
 - a. The experience and satisfaction a user have when interacting with an application
7. Metadata
 - a. Information about music content in the form of JSON that includes details like song title, artist, album, duration, and genre

7. Bibliography

- React Native Documentation

- OAuth Specifications
- Spotify Web API Documentation
- UI/UX Mobile Design Guidelines

8. Tutorial Screenshots

